🤮

SO intrebari

FIFO → PIPE CU NUME

PIPE → PIPE ANONIM

MMAP → apel de tip zero copy doarece intoarce un pointer catre zona alocata si doar il folosim

epoll - IO multiplexing

- ▼ Software stack
  - ▼ Ce este un apel de sistem?

    Un apel de sistem este o cerere a unei aplicații, care rulează în user-space în mod
    neprivilegiat, către sistemul de operare pentru efectuarea unei operații privilegiate, care va rula
    în kernel space. Este singurul mod prin care software-ul poate genera întreruperi.
  - ▼ De ce sunt necesare apeluri de sistem?

    Din motive de izolare, protejare, ca doar sistemul de operare să aibă acces în mod direct la resursele sistemului și pentru a permite acestuia sa controleze alocarea resurselor și modul în care acestea pot fi accesate.
    User space-ul nu are privilegiile de a realiza o operație din motive de securitate, astfel va face un apel de sistem pentru a executa operatii privilegiate precum lucrul cu memoria si comunicarea cu I/O -uri.
  - ▼ Ce avantaj / dezavantaje au apelurile de sistem?
    - Avantaj: mentin kernelul izolat de restul aplicatiilor, ruland in modul privilegiat, permitand accesul optim la resursele hardware (pentru care exista cerere mare din partea aplicatiilor);

- Dezavantaj: Overhead temporal ⇒ generat de tranzitia dintre user mode in kernel mode

▼ Ce înseamnă user/application mode/space (mod neprivilegiat)?

User mode este modul în care rulează, in general, un proces. Ele au un acces restrictionat la resursele sistemului, ruland intr-un spatiu neprivilegiat.

Kernel mode este zona specifica nucleului, unde se realizeaza operatii privilegiate fara a fi restrictionati in ceea ce priveste accesarea resurselor.

Separarea acestor zone ofera izolare/protectie sistemului, prevenind accesul neautorizat asupra resurselor.

▼ Cum se realizează tranziția din mod neprivilegiat în mod privilegiat?

- Tranziția se realizează prin intermediul interfetei expuse de sistemul de operare numita SYSCALL API si consta in invocarea apelurilor de sistem ce se realizeaza prin wrappere de biblioteca.
- Ceea ce ne ofera informatia despre mode-ul in care ne aflam in procesul curent este reprezentat de bitul de mod, care este 1 atunci cand ne aflam in user mode si 0 cand ne aflam in kernel mode. (este bitul procesorului si arata modul in care opereaza)

▼ Ce se întâmplă în momentul tranziției în mod privilegiat? Cum se/Cine asigură (enforcement) existența modului privilegiat?

- O aplicatie face o cerere, sistemul de operare verifica daca aplicatia are permisiunile necesare, apoi realizeaza actiunea ceruta si intoarce rezultatul.
- Distinctia dintre kernel space si user space este asigurata de modurile de privilegiu ale procesorului, marcate printr-un bit. Acesta este salvat in registrul CPL (Current Privilege Level) pe x86 sau CPSR pe ARM.

▼ Ce este o bibliotecă?

- Este o colectie de functii precompilate. Incorporeaza componente software reutilizabile. Atunci cand un program are nevoie de o functie, linker-ul va apela respectiva functie din biblioteca

▼ Care este diferenta dintre o aplicatie si o biblioteca?

- Biblioteca nu poate fi executata singura. O aplicatie, da.

- Totodata, biblioteca are o interfata generala pentru a fi utilizata pentru mai multe aplicatii, pe cand aplicatia ofera o implementare specifica.

▼ Care este asocierea apel de bibliotecă / apel de sistem?

- Un apel de biblioteca presupune zero (strcpy), unul (write) sau mai multe apeluri de sistem (popen). Exista apeluri de sistem care sunt invocate de mai multe apeluri de biblioteca (ex: fopen si open apeleaza syscall-ul open)

▼ Ce este entry point-ul intr-un executabil?

- Cand loaderul va incarca executabilul in memorie va seta valoarea instruction pointerul la valoarea din entry-point (entry-point = adresa primei instructiuni din cod)

▼ Care este rolul bibliotecii standard C (libc)?

- oferă o suită de funcții, atât pentru folosirea integrală în user space (precum `strlen()`), cât și pentru interacțiunea cu kernel-ul (wrapper-e precum `printf()`)

▼ Ce acțiuni se pot executa doar în mod privilegiat?

- Alocarea memoriei, interactiunea cu I/O uri, comunicarea cu alte procese, intreruperi

▼ Ce operații / instrucțiuni low-level (ISA) se pot executa doar în mod privilegiat?

- instructiunea `INT`: cheama o intrerupere software
- `HLT`: opreste procesorul pana la aparitia unei intreruperi sau a unui semnal

▼ Ce este un sistem de operare monolitic?

- Un sistem de operare monolitic este un sistem în care user services și kernel services sunt implementate în același spațiu de adrese (motiv pentru care este eficient). Este mai puțin flexibil, pentru ca atunci cand se face o modificarea asupra unei componente, trebuie schimbat tot.

▼ Ce este un sistem de operare de tip microkernel?

- Un sistem de operare de tip microkernel are user services si kernel services implementate în locuri diferite din memorie (motiv pt care este mai

lent, serviciile fiind nevoite sa comunice între ele).

▼ Care sunt avantajele unui sistem de operare monolitic?

- Avantaje: viteza de executie mare, overhead mic;
- Dezavantaje: securitatea potential mai slaba: intrucat kernelul este mai mare, si suprafata de atac este mai mare.
- Este mai putin flexibil, deoarece atunci cand se face o modificare asupra unei componente, trebuie schimbat tot

▼ Care sunt avantajele unui sistem de operare de tip microkernel?

- Avantaje: Daca un serviciu pica, nu afecteaza si alte servicii. Securitatea datelor este mai mare intrucat user space-ul este separat de kernel space.
- Dezavantaje: overhead temporal cauzat de tranzitia intre user mode si kernel mode

▼ Care tip de sistem de operare are mai multe apeluri de sistem?

- Cel de tip microkernel, deoarece sunt mai multe tranzitii user-kernel pentru a permite componentelor software care ruleaza in user mode sa interactioneze

▼ Care este avantajul folosirii mașinilor virtuale din perspectiva securității?

- Masinile virtuale au avantajul ca pot fi supuse unor malware fara a afecta sistemul de operare host.

▼ Ce este o bibliotecă statică? Ce este o bibliotecă dinamică?

- O bibliotecă statică este una ale cărei simboluri sunt copiate direct în executabilul programului, la link time.
- O bibliotecă dinamică este una ale cărei simboluri sunt aduse în executabil la load time, printr-un segment partajat.

▼ Când preferăm folosirea static linking, respectiv dynamic linking? Cu ce diferă un executabil dinamic de un executabil static?

- Executabilele statice: sunt portabile, pot fi transferate pe un alt sistem fara a fi nevoie de prezenta unor biblioteci. Lansarea lor in executie este mai rapida, nefiind nevoie de rezolvarea referintelor (dynamic binding): toate

referintele au fost rezolvate la link-time. Totusi, executabilul static e mai mare ca dimensiune, deoarece are toate bibliotecile incluse.

- Executabilele dinamice: ocupa mai putin spatiu pe disc, iar procesele aferente ocupa mai putina memorie fizica, partajand zonele read-only (.text, .rodata) din biblioteci cu alte procese obtinute din executabile dinamice care folosesc aceleasi biblioteci

▼ Dați exemplu de apel de sistem blocant

- read() → se blocheaza pana cand este citit cel putin un octet

▼ Dați exemplu de apel de sistem neblocant

- fork() → intoarce -1 daca da eroare, 0 pentru procesul copil si pid-ul procesului copil pt procesul parinte

▼ De ce, în general, o aplicație trebuie să execute un apel de sistem pentru a accesa un dispozitiv hardware? De ce NU poate accesa direct dispozitivul hardware?

- Pentru ca, in sistemele de operare care nu sunt unikernele, ruleaza in acelasi timp mai multe aplicatii. Majoritatea acestora doresc sa interactioneze cu hardware-ul, aparand astfel probleme cu accesul concurent la resurse. → sa se poate realiza accesul la resurse fara a se suprapune

▼ Ce intelegem prin overhead spatial si overhead temporal?

- spatial: dimensiunea suplimentara sau spatiu aditional necesar pentru realizarea unei anumite operatii (tabela standard de paginare aduce un overhead spatial intrucat facem corespondenta fiecarei pagini virtuale, fie ea valida sau nevalida → rezolvare: tabela de pagini ierarhica impartita pe directoare si apoi pe pagini)
- temporal: timpul suplimentar necesar pentru realizarea unei operatii (operatia de translatare a unei adrese virtuale intr-o adresa fizica aduce un overhead temporal intrucat este nevoie sa se acceseze de 2 ori memoria, o data pentru a accesa tabela de paginare si a gasi corespondentul paginii virtuale si a doua oara pentru a ajunge la adresa fizica gasita)

▼ Dati exemplu de mecanism / functie care reduce overhead-ul spatial si unul care reduce overhead-ul temporal.

- overhead spatial redus: tabela de paginare ierarhica
- overhead temporal redus: folosire TLB (memoria cache ce retine cele mai accesate pagini fizice cu scopul de a impiedica dublul acces in memorie)

▼ Ce inseamna double buffering? in ce situatie concreta (mecanism/-functie) apare?

- Este o tehnica prin care doua buffere sunt folosite pentru cresterea performantei. Astfel, procesare a unor date se face independent de aducerea altora: in timp ce se lucreaza cu datele dintr-un buffer, se pot pregati date in altul.
- Exemplu: SO vrea sa scrie date catre disk. Dar disk-ul este lent, asa ca, pentru a nu mai astepta dupa el, scrie intr-un buffer.. Problema: bufferul este plin si nu mai poate sa il foloseasca pentru altceva. Solutia? Double buffering: mai ia inca un buffer ca sa aiba unde sa scrie pana cand discul ia ce ii trebuie.

▼ Ce se intampla cand exista o eroare critica (de tip Segmentation fault) la nivelul sistemului de operare?

- Daca in nucleu se ajunge la Segmentation Fault, atunci SO intra in kernel panic, sistemul se opreste din a funciona si are nevoie de reboot pentru recuperare.

▼ Dați exemplu de un apel de bibliotecă standard C care nu cauzează apel de sistem.

- strcpy(), strlen() → acestea doar acceseaza memoria nu o si aloca

▼ Dati exemplu de un apel de biblioteca standard C care cauzeaza apel de sistem

- strdup() → aloca memorie pentru copierea unui string peste un altul

▼ Care este avantajul și dezavantajul existenței unei stive software la nivelul unui sistem de calcul?

- Avantaje: izolare buna a aplicatiilor, portabilitate, modularitate;
- Dezavantaje: overhead temporal

▼ Care este avantajul și dezavantajul folosirii unui limbaj de programare high-level (precum Java, Python, Lua)?

- Avantaje: management automat al memoriei, mai usor de utilizat, portabilitate mare;
- Dezavantaje: overhead temporal, control limitat asupra hardware-ului.

▼ Care este avantajul si dezavantajul folosirii unui limbaj de programare low-level (precum C, C++, Rust, D)?

- Avantaje: control mare asupra hardware-ului, rapiditate;
- Dezavantaje: prone to mistakes - mai dificil de inteles si de utilizat, dezvoltare mai lenta si mai putin productiva, portabilitate redusa

▼ Data

▼ Cum asigura sistemul de operare separatia intre procese?

- Prin mecanismul de memorie virtuala. Fiecare proces are impresia controlului intregii memorii. Astfel, ii este imposibil sa acceseze memoria altui proces

▼ Ce inseamna mecanismul de memorie virtuala?

- Este un mecanism utilizat de nucleul SO pentru a gestiona eficient memoria, prin izolarea intre procese si prin impartirea memoriei fizice intre acestea. Fiecare proces lucreaza cu memorie virtuala, motiv pentru care acesta are impresia ca are la dispozitie toata memoria.
- Dezavantaj al memoriei virtuale: overhead de calcul a adresei fizice din adresa virtuala la fiecare acces

▼ Ce reprezinta spatiul virtual de adrese al unui proces?

- Este o lista de zone de memorie in care sunt stocate date necesare procesului. VAS-ul are o dimensiune standard 2^{32} octeti(4 GB) si este

proprie fiecarui proces. Cuprinde zonele de text, data, rodata, bss, stack si heap.

▼ Cu ce difera zona de date de zona de cod in spatiul virtual de adrese al unui proces?

- Difera prin permisiunile lor, dar si prin functionalitate. Zona de date e RW- si retine, in general, variabilele utilizate de program. Zona de cod este R-X si retine instructiunile care se executa.

▼ Ce permisiuni are zona .text/.data/.rodata/.bss/de stivă/de heap?

- .text → read si execute
- .data → read si write
- .rodata → read only
- .bss → read si write
- stiva → read si write
- heap → read si write

▼ Ce este paginarea memoriei?

- Este mecanismul prin care RAM-ul este impartit eficient in mai multe bucati, numite pagini, de obicei de dimensiune 4kB. Pe un sistem de 32 de biti, un proces are 2^{20} pagini virtuale.
- Avantaje: nu mai avem fragmentare externa (cauzate de segmentare);
- Dezavantaje: poate aparea fragmentarea interna → ex dorim sa alocam 5kB ⇒ avem nevoie de 2 pagini ⇒ se vor aloca 8kB ⇒ avem o gaura de 3kB

▼ Ce rol are tabela de pagini?

- Are rolul de a face asocierea (maparea) dintre pagini virtuale si pagini fizice - este indexata cu adresa paginii virtuale si contine adresa paginii fizice.

▼ Ce este si ce rol are MMU (Memory Management Unit)?

- MMU sau MTU este o componenta hardware de pe procesor, ce are responsabilitatea translatarii paginilor virtuale in pagini fizice. El poate genera page fault-uri

▼ Ce rol are TLB?

- TLB = Translation Lookaside Buffer, este un cache pentru intrarile din tabela de pagini a unui proces. Pentru a micsora timpul de acces, acesta retine cele mai recent accesate intrari in tabela de pagini, si pentru fiecare adresa virtuala, verifica daca are adresa fizica memorata. Daca da, atunci are loc un TLB hit, altfel rezulta un TLB miss

▼ Care este ordinul de marime al numarului de intrari ale TLB?

- TLB-ul are intre 128 si 256 intrari

▼ Ce conține o intrare în tabela de pagini?

- PFN (page frame number) / adresa fizica;
- un bit ce precizeaza daca o pagina este valida; daca nu, rezulta in exceptie de acces la memorie(page fault);
- precizari legate de tipul de operatii permise (read-only, read-write);

▼ Ce înseamnă tabelă de pagini multi-nivel (ierarhică)? De ce este utilă?

- Este o tabela de pagini care retine adrese ale altor tabele de pagini. A aparut din nevoia de a micsora dimensiunea tabelei de pagini. O adresa din tabela ierarhica este formata din 10 biti (page directory index) + 10 biti (PTE index) + 12 biti (offset). O intrare in page directory refera o structura page table cu 210 intrari.

▼ Când are loc un TLB miss?

- Un TLB miss are loc atunci când pagina pe care se află adresa accesată nu a mai fost folosită, a fost folosită cu mult timp și a fost eliminată la update-ul TLB-ului sau când efectiv nu există o adresă fizică alocată pentru cea virtuală.

▼ De ce se golește TLB-ul (TLB flush) la schimbare de context?

- TLB flush are loc la context switch, deoarece TLB-ul trebuie sa retina doar informatia procesului curent. Nu este permis accesul proceselor la informatie ce nu ar trebui sa fie partajata. Dupa context switch, TLB este repopulat pe masura ce sunt accesate noi pagini

▼ De ce nu este nevoie de TLB flush la schimbarea de context între două thread-uri ale aceluiași proces?

- Pentru ca thread-urile aceluiasi proces impart aceeasi tabela de pagini, deci folosesc si au voie sa acceseze aceleasi adrese fizice.

▼ Ce înseamnă mecanismul de copy-on-write?

- copy-on-write este modalitatea prin care procesele copil primesc corespondența pagini - frames. Inițial, spațiul virtual de adrese al procesului copil point-ează către același spațiu fizic de adrese. Permisiunile sunt schimbate pe read only, și de abia la scriere, se duplică frame-urile, iar permisiunile sunt updatate.

▼ Dați exemple de situații în care are loc mecanismul de copy-on-write.

- virtualizare: COW este utilizat pentru a crea snapshot-uri ale sistemului virtual, astfel incat sa poata fi restaurat la o versiune anterioara in cazul unei erori. In acest caz, datele nu sunt copiate in mod real, ci sunt marcare pentru a fi copiate doar daca sunt modificate;
- sisteme de versionare ale fisierelor: COW poate fi utilizat pentru a pastra versiuni anterioare ale unui fisier, fara a fi necesara copierea acestuia in mod real la fiecare modificare. In acest caz, doar diferentele intre versiuni sunt stocate

▼ Cu ce apel de sistem asociem copy-on-write?

- fork()

▼ Când se duplică o pagină marcată copy-on-write?

- O pagina marcata copy-on-write se duplica cand un proces incearca sa scrie in ea.

▼ Cine detectează un acces de scriere într-o pagină marcată copy-on-write?

Handler-ul pentru page-fault al SO-ului detecteaza un acces de scriere intr-o pagina marcata COW.

El duplica apoi pagina fizica in cauza, actualizeaza permisiunile si referintele.

▼ Ce înseamnă demand paging?

- ”Demand paging” sau ”alocarea la cerere” reprezinta un mecanism prin care o pagina se poate ”rez-erva”. Atunci cand alocam o pagina virtuala noua (folosind malloc, mmap etc.) nu e obligatoriu sa alocam si un frame. Astfel, amanam alocarea efectiva a paginii fizice pana la primul acces: atunci se realizeaza si maparea. Intern, atunci cand se rezerva prima oara un page, se marcheaza pagina ca nevalida in tabela de pagini si ca valida-nemapata in informatiile sistemului de operare. La primul acces, MMU genereaza exceptie, deoarece intrarea in tabela de pagini este nevalida. Exceptia e capturata de SO, acesta alocand o pagina fizica pe care o completeaza apoi in tabela de pagini si marcheaza intrarea valida. Exceptia aruncata este numita page fault

▼ În ce situație apare page fault fără a cauza segmentation fault?

- in cazul demand paging, se aloca un page, dar nu si un frame: la aparitia unui page fault se va aloca un frame, dar nu cauzeaza segmentation fault;
- in cazul in care avem un page valid, dar al carui corespondent se gaseste pe disk, un acces genereaza page fault: continutul este swapped intr-un frame si procesul isi continua executia;
- in cazul copy-on-write, doua page-uri sunt marcare read-only si refera acelasi frame; in momentul in care unul dintre cele doua procese efectueaza o operatie de scriere, se genereaza page fault, se duplica frame-ul si se continua executia;

▼ Ce rol are spațiul de swap?

- Spațiul de swap are rolul de a reține paginile inactive din RAM, pentru a se putea face loc pentru altele noi, când memoria fizică este plină.

▼ Când are loc swap in și swap out?

- In momentul in care spatiul fizic (RAM-ul) devine insuficient, se evacueaza (swap out) anumite pagini fizice.
- In momentul in care aceste pagini sunt necesare, sunt readuse in memoria RAM (swap in)

▼ Care este rolul unui page fault? În ce condiții apare?

Page fault-ul este o exceptie generata de MMU, care apare atunci cand un proces incearca sa acceseze o pagina ce este marcata ”invalida” in tabela de pagini. Este de mai multe tipuri:

- minor: atunci cand pagina este in memorie, dar nu este mapata;
- major: atunci cand pagina este in swap;

▼ Care sunt secțiunile/zonele din spațiul de adrese al unui proces?

- .text, .rodata, data, bss, heap, biblioteci, stiva

▼ Ce secțiuni ale unui executabil se pot inspecta doar în timpul rulării?

- Zonele dinamice: stiva, heapul si bibliotecile dinamice

▼ Care sunt zonele writable din spațiul de adrese al unui proces?

- .data, stack, heap, .bss

▼ De ce sunt avantajoase bibliotecile dinamice pentru spațiul de adrese al unui proces?

- Pentru ca procesul va partaja zone de cod din cadrul bibliotecii dinamice cu alte procese. Ele sunt incarcate o singura data in RAM si apoi mapate in spatiul virtual al fiecarui proces care le foloseste

▼ Două procese sunt pornite din același executabil, ce zone din spațiul de adrese vor partaja?

- vor partaja .text, .rodata, bibliotecile dinamice si zona unde este mapat sistemul de operare (kernel space)

▼ Se alocă un buffer a[100]. De ce a[105] NU va rezulta, în general, în Segmentation fault?

- Pentru ca pentru a[100] se va aloca o pagina si a[105] nu va depasi spatiul unei pagini.

▼ În ce situație a[300] rezultă în Segmentation fault?

- Atunci cand lui a ii este alocat spatiu la finalul unei pagini, iar a[300] ar iesi din zona alocata.

▼ Câte pagini ocupa char a[100] pentru sistem standard cu pagini de 4096 de octeți?

- cel putin una si cel mult 2 (2 daca array-ul a este alocat la sfarsitul unei pagini si accesul la al 100-lea element ar depasi pagina)

▼ Câte pagini fizice alocă un apel mmap() care alocă 1MB? O pagină ocupă 4KB.

- Apelul mmap nu aloca pagini fizice, ci doar rezerva spatiu virtual.

▼ Câte pagini fizice alocă un apel calloc() care alocă 1MB? O pagină ocupă 4KB.

- Cel putin 250, cel mult 251 (cand doreste sa se aloca 1MB (1024*1024) se vor ocupa 1024*1024/4096 pagini)

▼ Ce înseamnă maparea unui fișier în memorie? De ce este avantajos să mapăm fișiere față de folosirea read/write?

- Este un mecanism prin care procesele acceseaza fisiere prin incorporarea lor directa in VAS. Adica, scrierea intr-o pagina virtuala conduce la scrierea in blocul corespunzator de pe disc al procesului. Are loc uzual pentru fisiere executabile si biblioteci partajate.
- Operatiile cu fisierele sunt acum operatii cu memoria, nu mai sunt apeluri de sistem read/write.
- Avantaje: overhead temporal scazut, deoarece nu se mai fac apeluri de sistem.
- Dezavantaje: fisierele trebuie sa aiba dimensiunea stiuta pentru mapare, nu se poate creste dimensiunea

▼ Câte page fault-uri se pot obține în cazul operației *a = b?

- Se poate obtine un pagefault daca pagina asociata instructiunii nu este mapata in memorie, apoi ar putea exista pagefault-uri de permisiuni: la adresa a nu se poate scrie, daca pagina asociata lui b nu a fost mapata.

▼ Care este numărul maxim de page fault-uri pe care îl poate genera expresia a = b + c?

- 1 daca nu a fost mapata pagina aferenta zonei .text

▼ Ce informații sunt reținute în stivă? Ce variabile C?

- Variabilele locale functiei, adresa de return, stack-frame-ul, parametrii functiei

▼ Ce se întâmplă la faza de loading (încărcarea unui executabil în memorie și crearea unui proces)?

- In timpul fazei de loading, sistemul de operare citeste executabilul de pe disk si mapeaza parti din el ın RAM. Tot acum se translateaza adresele logice ın adrese fizice.

▼ Ce înseamnă deturnarea fluxului de execuție a unui program (control flow hijack)? De ce este acest lucru relevant pentru un atacator?

▼ Ce înseamnă memory leak / memory disclosure? De ce este acest lucru relevant pentru un atacator?

- cineva neautorizat are acces date/adrese ale altuicuiva. Este util pentru atacator pentru a afla date personale sau alte portiuni de memorie de interes prin care poate ajunge la alte date/alte adrese sau pe care le poate suprascrie in vederea denaturarii fluxului.

▼ De ce în general, preferăm o împărțire a spațiului virtual de adrese între kernel space și user space? Și nu un spațiu dedicat pentru kernel space?

- Pentru ca zona kernelului este comuna intre procese. Cand se face context switch nu va fi nevoie sa se schimbe intrarile din page table asociate cu zona pentru kernel

▼ Ce secvență de cod C va duce la o excepție de acces la memorie (de tip Segmentation fault)? De ce?

- Dereferențierea unui pointer null. Acesta este reprezentat ca o referință către adresa 0 din adress space, iar, prin urmare, MMU-ul indică ca nefăcând parte din memorie pagina care o conține. Pagina nu este, astfel, inclusă în VAS, iar o posibilă scriere sau citire pe aceasta rezultă într-un acces invalid și un SEGFAULT.

▼ Cu ce diferă o funcție de o variabilă într-un executabil și/sau în cadrul spațiului de adrese al unui proces?

O variabilă nu este executabilă, ea se află în zona .data, .rodata, .bss sau stivă al spatiului de adrese.

O funcție este un simbol ce poate fi executat. Ea se află în zona .text și are permisiuni de citire și execuție.

▼ Două procese partajează o zonă de memorie. Cum se manifestă acest lucru în tabelele de pagini ale celor două procese?

- Cele două tabele de pagini vor avea anumite intrări comune, reprezentând adresele fizice pe care cele două procese le folosesc concomitent.

▼ Putem avea mai multă memorie fizică decât dimensiunea maximă a spațiului virtual de adrese al unui proces? Dar invers?

- În general, memoria virtuală este mai mare sau egală cu cea fizică. Aceasta abstractizare a memoriei este necesara atunci cand programul necesita mai multa memorie decat este valabila in spatiul fizic. Atfel, datele vor fi stocate temporal intr-un VAS.

▼ Ce zone de memorie se aloca static? Dar dinamic?

- Zonele de memorie care se aloca static cand se da load la executabil sunt: .text, .rodata, .data, .bss
- Zonele de memorie care se aloca dinamic sunt: stack, heap si bibliotecile dinamice

▼ Ce se întâmplă cu o variabilă modificată într-un proces copil din perspectiva procesului părinte?

- Procesele nu impart zona .data ⇒ daca o variabila din procesul copil va fi modificata, cea din procesul parinte nu va suferi nicio schimbare

▼ Ce reprezintă un loader? Ce rol are acesta?

- Loader-ul este o componentă a sistemului de operare care se ocupă cu încărcarea programelor și a bibliotecilor. Are rolul de a valida permisiuni, de map-are a executabilului de pe disk în memorie, de a copia argumentele în memoria virtuală, inițializează regiștrii (spre exemplu, `SP`) și face jump la entry-point (`_start`).

▼ La ce folosim apelul mprotect()? La ce mecanism de securitate putem face bypass folosind acest apel?

▼ În ce zonă de memorie se află o variabilă globală, inițializată cu valoarea 0?

.bss

▼ Calcul

▼ Cum modifica fork() si exec() spațiul virtual de adrese?

- Atât `fork()` cât și `exec()` creează un spațiu nou de adrese, diferența dintre cele fiind aceea că `fork()` creează un nou spațiu de adrese pentru procesul copil, pe când `exec()` golește spațiul virtual de adrese al procesului curent (mai puțin partea de kernel space) și regiștrii și pregătește pentru a rula noul program.

▼ Ce efect are apelul exec()?

- Apelul exec are rolul de a schimba imaginea procesului curent si apoi invoca loaderul pentru a lansa spre executie noul proces.

▼ Câte threaduri se pot găsi în starea RUNNING, READY și WAITING?

- RUNNING → cate coruri avem
- READY si WAITING → oricate in limita resurselor sitemului

▼ Ce este o schimbare de context? Ce se întâmplă la o schimbare de context?

- Se referă la schimbarea unui proces care rulează pe un procesor (este în starea
  RUNNING) cu un alt proces (aflat în starea READY). Este necesară pentru a asigura folosirea
  optimă a procesorului (dacă un proces se blochează îi ia altul locul) și pentru asigurarea echității
  (fairness) a sistemului (procesele se schimbă cu altele pentru a permite câtor mai multe să ruleze
  în sistem).
  Schimbare de context intre P1 si P2 : P1 salveaza starea in PCB ul lui, P2 o restaureaza pe a sa.

▼ Ce cauzează schimbări de context?

- 5 conditii principale care iti genereaza context switch:
  1) RUNNING → READY - schimbare voluntara (yield)
  2) RUNNING → BLOCKED (waiting) - schimbare voluntara, o actiune executata explicit de
  procesor are nevoie de resurse (operatii blocante sau I/O o declanseaza)
  3) RUNNING → READY - schimbare nevoluntara (apare un proces candidat mai bun, prioritar)
  4) RUNNING → READY - schimbare nevoluntare (eg expira cuanta)
  5) RUNNING → TERMINATED - aici depinde ce fel de schimbare e dupa cum s-a terminat.

▼ Ce este o schimbare de context voluntară și o schimbare de context nevoluntară?

- O schimbare de context voluntara este aceea in care procesul face o actiune care stie ca va avea drept consecinta scoaterea lui de pe procesor: yield sau se blocheaza in asteptarea I/O ului)
- O schimbare de context involutara nu este data de actiunea directa a procesului, ci aceasta se face atunci cand unui proces ii expira cuanta de timp sau un alt proces are o prioritate mai mare

▼ Ce sunt threadurile cu implementare user-level și thread-urile cu implementare kernel-level?

- Thread-urile user-level (sau green threads) sunt thread-uri ce rulează exclusiv în user space, fiind controlate de biblioteci; pentru sistemul de operare, ele sunt privite ca un singur thread, putând folosi la un moment dat un singur procesor.
- Thread-urile de sistem sunt thread-uri ce rulează în kernel space (produc context switch-uri) și sunt privite ca mai multe acțiuni ce pot fi paralelizate (ocupă mai mult de un core).

▼ În ce situație este utilă zona TLS (thread local storage)?

- TLS-ul se afla la baza stivei fiecarui thread si are rolul de a retine variabilele globale ale threadului, dar locale per thread

▼ De ce este necesară sincronizarea proceselor/thread-urilor?

- Sincronizarea este folosită pentru a asigura două comportamente din partea thread-urilor: serializare și acces exclusiv. Prin acestea, este menținut caracterul determinist al acțiunilor care pot fi deturnate de la cursul lor natural din cauza thread-urilor sau proceselor modificând zone de memorie partajată.

▼ Ce înseamnă race condition?

- Race-condition se referă la rezultatul nedeterminist pe care o are acțiunea făcută două sau mai multe thread-uri simultan, dorind să modifice o resursă comună.
- Race condition se realizeaza atunci cand outupul programului depinde de ordinea executiei threadurilor.

▼ Ce înseamnă deadlock?

- Deadlock reprezinta actiunea in care 2 threaduri blocate asteapta informatii/resurse din partea celuilalt.

▼ Care sunt dezavantajele sincronizării?

- implementarea ineficienta: putem sa avem deadlockuri, race conditions
- overhead temporal mare: cand se face lock un singur procesor acceseaza memoria, la spinlock se face busy waiting ocupand loc pe procesor, iar la mutex/semafor: se fac apeluri de sistem si context switchuri

▼ Ce înseamnă TOCTOU (time of check to time of use)?

- Se refera la intervalul de timp dintre momentul cand o resursa este marcata ca fiind utilizata pana cand threadul o va putea utiliza. Aceasta poate fi critica in creearea de race conditions deoarece o resursa poate fi schimbata intre timpul de check si use ale altui proces.

▼ Când se blochează un producător în problema producator-consumator? Dar un consumator?

- Un producător se blochează dacă consumatorul nu a citit informația pe care el a produs-o (când buffer-ul comun e plin).

- Un consumator se blochează dacă producătorul nu i-a oferit nicio informație pe care el să o poată folosi (când buffer-ul comun e gol).

▼ De ce este necesară prezența unei instrucțiuni de tipul atomic_compare_and_swap în fiecare ISA?

- Aceasta instrcutiune sta la baza sincronizarii doarece daca aceasta nu ar fi atomica, magistrala nu ar fi inchisa si mai multe coruri ar putea avea acces la aceeasi informatie simultan.

▼ Cu ce diferă un spinlock de un mutex? Când folosim spinlock-uri? Când folosim mutex-uri?

- Spinlock-urile sunt o formă de busy waiting în starea de RUNNING. Ele nu determină ieșirea de pe procesor a thread-ului care așteaptă. Sunt folosite atunci când timpul de așteptare nu este mare (zone critice mici, fără apeluri blocante), iar un context switch este mai costisitor în comparație.
- Mutex-urile sunt o formă de sincronizare care funcționează prin context switch, și ocazional apel de sistem (chemarea scheduler-ului la `lock()` și `unlock`). Sunt folosite în toate celelalte cazuri în care spinlock-ul nu este viabil: apeluri blocante, zone critice mari
- Se vor utiliza spinlockuri in loc de mutexuri atunci cand resursele utilizate de catre un thread vor fi eliberate intr-un timp relativ scurt si nu e nevoie sa blocam threadul care are nevoie de acele resurse.

▼ Ce efect are folosirea operatorului & din shell în crearea unui proces?

- Shell-ul nu mai face waiting blocant pentru procesul ce primește &. Acesta este trecut în background, își continuă execuția, iar la final, când dă exit, părintele (shell-ul) primește semnalul `SIGCHLD` și face `wait()`.

▼ Ce este un proces zombie? Cum apare un proces zombie? Care este problema proceselor zombie?

- Un proces zombie este un proces care și-a încheiat execuția și căruia nu i s-a făcut wait de către părintele său. Problema cu procesele zombie este că ocupă resurse utile (PID, intrare în tabela de procese) degeaba, deoarece acestea nu mai au nimic de executat, dar nu sunt eliberate.

▼ Ce este un proces orfan? De ce un proces este orfan foarte puțin timp?

- Un proces orfan este procesul caruia i-a murit tatal, insa acesta va fi orfan pentru foarte putin timp deoarece va fi adoptat de catre procesul init.

▼ Poate fi un proces zombie orfan? Ce se întâmplă cu un proces zombie orfan?

- Un proces este zombie orfan cand copilul si-a incheiat executia si parintele nu l-a asteptat, iar intre timp parintele a murit. Astfel, copilul zombie orfan va fi preluat de catre init si i se vor elibera resursele utilizate.

▼ Ce se întâmplă dacă un thread realizează un acces invalid la o zonă de memorie?

- Thread-ul, împreună cu întreg procesul de care ține, va primi SEGFAULT, iar programul se va încheia cu o eroare.

▼ Poate un thread să acceseze stiva altui thread? Cum?

- Da, se poate realiza accesul catre stiva altui thread intrucat aceasta nu este izolata. Se poate accesa stiva printr-o variabila globala pe care o mostenesc toate threadurile, modificarile facute de catre un thread vor fi reflectate asupra tuturor threadurilor.

▼ De ce schimbarea de context între două thread-uri ale aceluiași proces este mai rapidă decât schimbarea de context între două thread-uri din procese diferite?

- Pentru că thread-urile aceluiași proces share-uiesc majoritatea VAS-ului procesului, iar flush-ul TLB-ului nu este necesar

▼ Ce limitează numărul maxim de threaduri care pot fi create în cadrul unui proces?

- Dimensiunea memoriei virtuale, deoarece pentru fiecare nou thread trebuie creată o nouă stivă în cadrul VAS-ului procesului.

▼ Ce limitează numărul maxim de procese care pot fi create în cadrul unui sistem de calcul?

- Numărul maxim de intrări în tabela de procese. De obicei un număr fix setat de sistemul de operare

▼ Ce se întâmplă când toate procesele sistemului sunt blocate?

- Pe fiecare procesor se planifica un proces special numit IDLE (care face busy waiting) daca nu exista alt proces care poate fi planificat.

▼ Ce înseamnă waiting time (timp de așteptare) în planificarea proceselor?

- Waiting time este impul pe care îl petrec procesele în starea READY

▼ Putem avea un sistem multi-core cu un singur proces aflat în starea RUNNING și mai multe procese în READY?

- Da, daca threadurile aceluiasi proces ocupa toate corurile.

▼ Ce înseamnă starea READY/RUNNING/TERMINATED/WAITING(BLOCKED)?

- READY → thread-ul este gata pentru a intra pe procesor, așteaptă să i se facă loc, să fie planificat
- RUNNING → thread-ul rulează, are o cuantă stabilită și poate executa oricâte acțiuni pe parcursul ei
- TERMINATED → thread-ul și-a încheiat execuția și face `exit()`
- BLOCKED → thread-ul a făcut un apel blocant și așteaptă după I/O

▼ Ce este un proces I/O intensive?

- Un proces care face multe apeluri blocante, comunică des cu diskul sau placa de rețea; trece des din RUNNING în BLOCKED.

▼ Ce este un proces CPU intensive?

- Un proces care face calcule intense pe procesor si care nu se blocheaza foarte des, fiind de cele mai multe ori scos involuntar de pe procesor.

▼ Cum tratează planificatorul procesele I/O intensive și procesele CPU intensive?

- Procesele I/O intensive sunt prioritare: ele, în mod garantat, fac multe apeluri blocante și ajung voluntar în BLOCKED, nu au tendința de a acapara CPU-ul. Totdata, pentru procesele CPU intensive le este acordata o cuanta de timp mai mica pentru a pastra waiting timeul cat mai mic si impartirea resurselor cat mai eficienta.

▼ Două thread-uri ale unui proces execută aceeași funcție. Care sunt diferențele între cele două thread-uri?

- Fiecare thread are propria lui stiva (pot diferi stack pointerul si instruction pointerul). Totodata, poate diferi rapiditatea fiecarui thread, rapiditatea accesului la resurse.

▼ Cu ce diferă procesul copil (creat prin fork) de procesul părinte?

- Difera PID-ul, au VAS-uri diferite desi pointeaza catre aceeasi zona de memorie fizica si au acelasi continut.

▼ Cu ce diferă un proces zombie de un proces orfan?

- Un proces zombie este atunci cand procesul copil si-a incheiat executia si procesul parinte nu l-a asteptat, iar un proces orfan este acela caruia i-a murit tatal.

▼ Ce parametri ai planificatorului trebuie să modificăm pentru a avea un sistem cu productivitate mai mare?

- Procesele CPU intensive sa primeasca o cuanta de timp mai mare.

▼ Ce parametri ai planificatorului trebuie să modificăm pentru a avea un sistem cât mai interactiv (responsive)?

- Procesele IO intensive sa fie prioritare.

▼ Am avea nevoie de folosirea unui apel de sistem pentru crearea unui thread în cazul unei implementări de tip user-level threads? Dar în cazul deschiderii unui fișier în același scenariu

- Nu este necesar un syscall pentru crearea unui thread în user space, deoarece există conceptul de green threads (fire de execuție ce rulează la nivel de bibliotecă).
- Este necesar un syscall pentru deschiderea unui fișier, deoarece, în acest caz, interacționăm cu hardware-ul (disk-ul) și avem nevoie de permisunea kernel-ului pentru a face asta.

▼ Ce se întâmplă dacă folosim apeluri de sistem blocante în interiorul unui spinlock?

- Deadlock. Sa zicem ca apelul de sistem blocant necesita acces la acelasi resursa partajata protejata de catre spinlock ⇒ ambele vor astepta resurse si nu le vor primi.

▼ De ce este necesară folosirea prefixului LOCK pentru realizarea operațiilor atomice?

- Prefixul LOCK anunță sistemul de operare să facă un lock hardware pe magistrală. Un singur core mai are acces, astfel, la memorie, iar operația atomică poate avea loc.

▼ Care este avantajul folosirii mutexurilor în defavoarea spinlockurilor și invers?

- Mutex-urile sunt folosibile pentru critical regions mari, cu apeluri blocante, pe când spinlock-urile sunt ieftine când vine vorba de `lock()` și `unlock()`, pentru că nu fac context switch și nici nu invocă scheduler-ul.

▼ Care este echivalentul apelului wait() pentru threaduri?

- join() → un thread parinte asteapta un thread copil sa-si termine executia

▼ De ce este nevoie de apelul pthread_wait() / pthread_join()?

- Apelurile `wait()` / `pthread_join()` realizează eliberarea resurselor folosite de proces / thread după terminarea acestora.

▼ Ce tranziție între stările unui thread nu este posibilă?

- READY → BLOCKED: threadul trebuie sa faca o actiune pentru a face un apel blocant, iar asta se intampla doar cand threadul este in starea running.

▼ I/O

▼ Ce conține un FCB (File Control Block)?

- Identificator, permisiuni, ownership, size, timestamps, numarul de referinte si pointer catre date

▼ Ce reprezintă un descriptor de fișier?

- Descriptorul de fișier este un indice în tabela de file descriptors. Ca intrare pentru un descriptor de fișiere este reținut un pointer către un open channel structure.

▼ Ce reprezintă tabela de descriptori de fișiere?

- ca structură de date este
  un vector de pointeri. Acești pointeri referă structuri de fișier deschis de proces. Când un proces
  deschide un fișier, se alocă o structură de fișier deschis, iar adresa acestei structuri este stocată
  într-un loc liber (indicat de descriptorul de fișier) din tabela de descriptori de fișier.

▼ Câte tabele de descriptori de fișiere se găsesc într-un sistem de operare?

- Una per proces.

▼ Ce efect are apelul dup()?

- Apelul lui dup are rolul de a duplica un file descriptor peste urmatorul file descriptor disponibil.

▼ Ce efect are apelul close()?

- Eliberează file descriptor-ul asociat. Scade numărul de link-uri către un open file structure. Dacă numărul ajunge să fie 0, atunci open file structure-ul este eliberat la rândul său.

▼ Ce apeluri modifică pointer-ul/cursorul de fișier (file pointer)?

- open, read, write, seek

▼ Ce apeluri modifică dimensiunea fișierului?

- write, truncate, open

▼ Ce efect are apelul/comanda truncate?

- truncate are rolul de a schimba dimensiunea fisierului → daca se scade se pot pierde date, daca se mareste e ok

▼ Care sunt tipurile de fișiere pe un sistem de fișiere uzual Unix?

- Regular, directory, symbolic link, FIFO special, block special, character special si socket.

▼ Ce este un sistem de fișiere virtual?

- Fiecare fisier vine cu propria sa implementare si s-a dorit crearea unui layer abstract pentru operatiile sistem fisier ca sa nu se abordeze in mod diferit lucrul cu fiecare fisier.
- VFS defineste o interfata generica pentru operatiile sistem-fisier. Toate programele care lucreaza cu fisiere folosesc operatiile puse la dispozitie de aceasta interfata.
- VFS expune operatii precum: create, read, write, seek, close, mmap.

▼ Ce este un dispozitiv virtual?

- Un dispozitiv virtual este o reprezentare a dispozitivului fizic care pointeaza catre un device file ce nu are un hardware asociat, kernelul identificandu-l drept un periferic, insa acesta este doar un fisier.

▼ Ce tipuri de dispozitive cunoașteți? Clasificați-le din orice punct de vedere cunoașteți

În funcție de câți bytes gestionează:

1. char devices (se ocupă cu un singur caracter la un moment dat - dispozitive seriale): tastatură, mouse, terminal.
2. block devices (se ocupă cu blocuri de date cu dimensiunea multiplu de 512 - random acces devices): disk, CD-ROM, flash drives

▼ Cu ce diferă un dispozitiv de tip bloc de un dispozitiv de tip caracter? Dați câte un exemplu de fiecare

1. char devices: pentru char devices nu se face buffering; read și write există ca funcții și sunt blocante; au un driver mai simplu, pentru ca sunt de tip stream și nu au de reținut decât o singură poziție (cea curentă).
2. block devices: block devices sunt accesate prin cache; read și write sunt asincrone; au un driver complicat, pentru că acesta trebuie să treacă prin buffer cache pentru a scrie și citi.

▼ De ce nu are sens operația de seek pe un dispozitiv de tip caracter?

- Pentru că dispozitivele de tip caracter sunt de tip stream și reține un singur byte la un moment dat.

▼ Ce adresă IP locală și ce port local are un socket întors de apelul accept()?

- Apelul `accept()` întoarce un fd pentru un socket cu adresa IP a rețelei locale și portul aplicației care dorește să se conecteze.

▼ Ce valoare poate întoarce un apel read() sau un apel write()?

- `read()` întoarce numărul de bytes citiți, 0 pentru EOF sau -1 în caz de eroare.
- `write()` întoarce numărul de bytes scrișI sau -1 în caz de eroare.

▼ Ce operații se pot face pe fișiere?

- open, read, write, seek, truncate, close, create, delete

▼ Ce operații asupra fișierelor modifică/nu modifică valoarea cursorului unui fișier?

- seek, open, read, write
- nu modifica: ftruncate()

▼ Ce operații asupra fișierelor modifică/nu modifică dimensiunea fișierului?

- open, write, truncate
- nu modifica: lseek, read

▼ Unde este reținută valoarea cursorului de fișiere (file pointer) și unde este reținută dimensiunea fișierului?

- Cursorul este retinut in structura de fisier deschis, iar dimensiunea este in FCB.

▼ De ce avem două buffere asociate fiecărui socket, ce rol are fiecare?

- Socket-ul, spre deosebire de pipe, permite comunicarea bidirecțională. Pentru a asigura faptul că sender-ul și receiver-ul nu trimit informație dintr-o parte în alta în același timp, pe același canal, există două buffere: unul de send și altul de receive.

▼ Ce este o operație asincronă?

- Operatiile asincrone au drept caracteristica faptul ca pot fi executate in alta ordine fata
  de cea in care apar in cod.

▼ Ce este o operație neblocantă?

- O operație neblocantă este caracterizată de faptul că se întoarce imediat, indiferent dacă a finalizat acțiunea sau nu. Spre exemplu, un `read()` neblocant ar citi direct (dacă ar avea ce) sau s-ar întoarce cu un mesaj de eroare special, care anunță faptul că nu și-a dus la bun sfârșit task-ul, pentru că s-ar fi blocat.

▼ Ce întoarce o operație asincronă?

- Operațiile asincrone nu întorc date, ci un cod de eroare, 0 sau 1. Datele pot fi accesate printr-un `eventfd()`

▼ Ce întoarce o operație blocantă?

- Operația blocantă întoarce întotdeauna rezultatul acțiunii; tocmai de aceea este blocantă, pentru că așteaptă mereu până la încheierea task-ului

▼ Cu ce diferă un socket de rețea de un socket UNIX?

- Socket-ul de rețea este folosit pentru comunicarea proceselor de pe sisteme diferite. Socket-ul UNIX este o formă de comunicare între procese aflate pe același sistem.

▼ Care este diferența între un pipe anonim și un pipe cu nume (named pipe)?

- Pipe-ul cu nume permite comunicarea între două procese neînrudite, pe când prin pipe-ul anonim se poate transmite informație doar între procese înrudite. Mai mult, pipe-ul cu nume apare ca o intrare în sistemul de fișiere și reprezintă chiar un tip de fișier (FIFO).

▼ Ce este buffer cache-ul? Care este rolul său?

- Buffer cache-ul este un buffer de blocuri aflat în kernel space. Acesta reține blocurile de date folosite recent, pentru a scădea numărul de interacțiuni cu disk-ul, o componentă extrem de lentă față de memorie.

▼ De ce operația write pe fișiere este foarte rar blocantă?

- Operația de `write()`ar fi blocantă dacă buffer-ul din kernel space ar fi plin. Acesta este extrem de rar plin, pentru că dacă se umple, este fie eliberat (prin scrierea pe disk a informațiilor pe care le reține), fie mărit.

▼ În ce situație operația read() pe fișier se blochează?

- Operația de `read()` este blocantă în cazul în care buffer-ul din kernel space este gol. Acțiunea va fi blocată până când cel puțin un octet ajunge în buffer.

▼ Care este rolul unui device driver?

- Device driver-ul este o bucată de software folosită pentru a controla diferite componente hardware. Are rolul de a permite sistemului de operare să se folosească de periferice.

▼ Ce rol are controller-ul hardware?

- Controlează componentele hardware (are rol de administrator). Este la rândul său o componentă hardware care permite transmiterea, flow-ul de date, între mai multe entități fizice.

▼ Ce înseamnă zero-copy? Ce mecanism/apel folosește zero-copy?

- Zero-copy se referă la transferul de date între mai multe buffer-e din kernel space fără a se trece prin user space. Interacțiunea cu aplicația devine minimă, iar costul temporal adus de domain switch este evitat.
- Apeluri folosite pentru zero-copy sunt `sendfile()`, `splice()`

▼ Ce garanții ni se oferă în momentul în care apelul send() se întoarce în user space?

- Că datele în întregime sau doar o parte (de aceea `send()` trebuie făcut într-un loop pentru a asigura că s-au trimis toate datele) au fost copiate într-un buffer de send.

▼ Cu ce diferă afișarea folosind printf() față de folosirea write()?

- printf() spre deosebire de apelul write() va mentine un buffer in user space caruia i se va da flush si trimis spre kernel cand se da newline. (sau la apel explicit de flush()). write() va face apel de sistem pentru fiecare litera

▼ De ce subsistemul de networking nu folosește buffer cache-ul?

- Pentru că buffer cache-ul este un cache exclusiv pentru lucrul cu disk-ul. Blocurile de date cele mai frecvent folosite sunt aduse în kernel space (în acest buffer) și accesate direct de aici, deoarece interacțiunea cu disk-ul este scumpă din punct de vedere al timpului (memoria persistentă e lentă).

Astfel, buffer cache-ul nu ar avea sens pentru subsistemul de networking care funcționează pe bază de pachet, ca unitate fundamentală de comunicare.

▼ Ce rol are apelul / comanda sync?

- `sync()` realizează sincronizarea dintre buffer-ele din kernel space cu ceea ce rezidă pe disk

▼ Care este rolul apelului ioctl / DeviceIoControl?

▼ De ce în general doar utilizatorul root are permisiuni de scriere (uneori doar root are permisiuni de citire) pe intrările din /dev?

- Pentru ca accesul direct la hardware se face doar din modul privilegiat
- Daca oricine ar putea face modificari pe modul de functionare ale device driverelor, atunci ar
  putea aparea probleme de securitatea, se pot crea vulnerabilitati si comunicarea cu
  dispozitivele de I/O ar fi nefunctionala.

▼ De ce este apelul fwrite() mai rapid decât write atunci când facem multe scrieri?

- fwrite() retine un buffer in care scrie si caruia ii da flush si trimis catre kernel space cand isi termina crierea, pe cand write face syscall pentru orice actiune de scriere.

▼ Ce se întâmplă dacă facem open de mai multe ori consecutiv pe același fișier?

- Se va adăuga un nou file descriptor în FDT și se va crea un alt open file structure pentru fd-ul creat

▼ Cum se modifica tabelul de file descriptori după open() si dup()? Este echivalent cu doua apeluri open()?

- `open()`
  creează un nou file descriptor și un nou open file structure în FDT, pe când `dup()`
  creează un nou file descriptor și point-ează către același open file structure de la vechiul fd, pe care îl primește ca argument, incrementând numărul de referințe (link-uri) din structură. `dup()`
  nu este neapărat echivalent cu două apeluri `open()`

pentru că **nu**
creează un open file structure; el doar copiază referința dintr-o parte a vectorului de fd-uri în alta.

▼ Ce fișiere sunt deschise, în general, la crearea unui proces nou?

0,1,2 → stdin, stdout, stderr

▼ De ce este utilă prezența unor dispozitive pur virtuale în ierarhia /dev (ex. */dev/vboxnetctl*, */dev/urandom*)?

- Pentru a putea emula un tip de dispozitiv hardware fără a avea la îndemână piesa respectivă. Pentru a putea folosi functionalitati precum cele date de urandom si random.

▼ De ce, în general, apelul write() pe un fișier nu blochează threadul curent?

- Pentru ca se face in mod asincron, se scrie in bufferul din user space si apoi se face memcpy in cel din kernel space.

▼ În ce situație un apel read() pe un fișier blochează threadul curent?

- atunci cand bufferul din kernel space este gol

▼ De ce spunem că operațiile read() / write() seamănă cu operația memcpy()?

- Ambele scriu/citesc dintr-un buffer pe care apoi il copiaza peste un altul.

▼ Interacțiunea între aplicații

▼ Ce forme de comunicare inter-proces cunoști?

- comunicare prin pipe-uri, socketuri, fisiere, shared memory
- sincronizare: semafoare cu nume
- semnale

▼ Ce este un semnal? Când se trimite un semnal către un proces?

- Semnalul este un mesaj trimis asincron proceselor pentru a declanșa un anumit comportament. La primirea semnalului este rulat un handler specific care tratează eroarea, întrerupe, suspendă sau omoară procesul.

▼ Cine trimite un semnal unui proces?

- Sistemul de operare, alt proces sau chiar el însuși.

▼ Cum este implementat operatorul | din shell?

- este implementat similar cu `pipe()` din C: este folosit un buffer comun în care primul proces trimite informație, iar cel de-al doilea o primește. Cele două capete ale buffer-ului corespund celor două procese care comunică.

▼ Care sunt avantajele și dezavantajele folosirii memoriei partajate pentru comunicarea inter-proces?

- Avantaj: Mai putina zona de memorie folosita; mai rapida comunicarea prin memorie decat prin mesaje/socketi
- Dezavantaj: Sincronizarea, in lipsa ei se pot genera date incoerente

▼ Care sunt avantajele și dezavantajele pipe-urilor pentru comunicarea inter-proces?

- Avantaje: nu este nevoie de sincronizare, comunicarea se face unidirectional
- Dezavantaje: au o comunicare unidirectionala, daca bufferul este plin procesul ramane in starea blocked pana se elibereaza memoria si nu se pot mapa fisiere prin pipe

▼ Care este limitarea folosirii pipe-urilor anonime?

- Pot întreține comunicarea doar între procese înrudite. De asemenea, sunt dependente de procesele asociate.

▼ Care este diferența principală între FIFO și socket UNIX?

- FIFO-urile au un singur canal de comunicare (unidirecțional), pe când socket-urile UNIX prezintă două buffer-e (comunicare bidirecțională), unul pentru send și unul pentru receive

▼ Ce primitive de interacțiune pot fi folosite între procese de pe sisteme diferite?

- socketuri de retea

▼ Ce primitive de comunicare pot fi folosite doar între threaduri?

- shared memory

▼ Ce primitive de comunicare pot fi folosite doar între procese înrudite?

- pipe uri anonime

▼ Cum pot două procese să folosească semafoare anonime?

- Două procese înrudite pot folosi același semafor anonim dacă la crearea acestuia este pus 1 pentru argumentul de `pshared`

▼ Cum pot două procese neînrudite să folosească semafoare anonime?

- Două procese neînrudite pot folosi același semafor anonim dacă acesta este plasat într-o zonă de memorie pe care procesele o share-uiesc.

▼ Ce rol are apelul mmap() în cazul partajării memoriei între două procese?

- creeaza zone de memorie anonime ce pot fi folosite de procesele inrudite.

▼ Ce mecanisme putem folosi pentru a asigura accesul exclusiv la o zonă de memorie partajată între două procese?

- prin mutexuri (lock pe o resursa din memoria partajata) sau semafoare

▼ De ce este nevoie de sincronizare la folosirea memoriei partajate dar nu la folosirea unui socket pentru comunicarea între două procese?

- La folosirea unui socket, fiecare proces are un buffer in care transmite date si unul in care primeste, deci cele 2 nu pot accesa bufferul celuilalt. La memoria partajata intre 2 procese avem nevoie de sincronizare pentru ca 2 procese pot accesa aceeasi zona de memorie si sa o modifice simultan ⇒ coruperea datelor

▼ Ce înseamnă aplicație omogenă? Dați un exemplu

- O aplicație omogenă folosește thread-uri/procese pentru a realiza același tip de acțiune concomitent
- ex: multi-threaded web servers, paginile din firefox(threaduri), paginile din chorme (procese)

▼ Ce înseamnă aplicație eterogenă? Dați un exemplu.

- O aplicație eterogenă folosește thread-uri/procese pentru a realiza tipuri diferite de acțiuni concomitent. Interactiune mare intre threaduri/procese ⇒ folosite in IPC
- ex: gitlab

▼ Care este avantajul și dezavantajul distribuirii unei aplicații pe mai multe sisteme?

- Avantaje: se evită încărcarea sistemului propriu, putere computațională mai mare (sisteme distribuite).
- Dezavantaje: comunicare mai greoaie între componente (se pot folosi doar sockets de rețea, mai scumpi ca timp)

▼ Care este avantajul implementării unei aplicații în format multi-process față de format multi-threaded?

- Securitate, procesele au un vas propriu, pe cand threadurile il au comun
- Robustete: daca un proces crapa, nu crapa tot programul (cum s-ar intampla la threaduri)

▼ Cum pot două procese partaja memorie anonimă? Adică folosind flagul MAP_ANONYMOUS la mmap().

- Folosirea flagului MAP_ANONYMOUS impreuna cu MAP_SHARED are urmatoarele efecte:
  - maparea si zeroizarea unei zone distincte de memorie
  - copiii mostenesc zonele mapate ale parintilor
  - nu exista copy on write cand unul dintre procese incearca sa scrie
  - zonele mapate anonim sunt folosite ca mecanism de IPC, ceea ce numim shared memory

▼ Ce mecanism de interacțiune între aplicații au un buffer și ce mecanisme de interacțiune nu au un buffer?

- cu buffer: pipes, sockets
- fără buffer: fișiere, shared memory, mutex-uri, semafoare, semnale